INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Title: Command Injection

Lab 3: Command Injection – Short Report

Objective

To understand command injection, exploit it in a vulnerable web app, and fix it using basic input sanitization.

Task 1: Exploitation

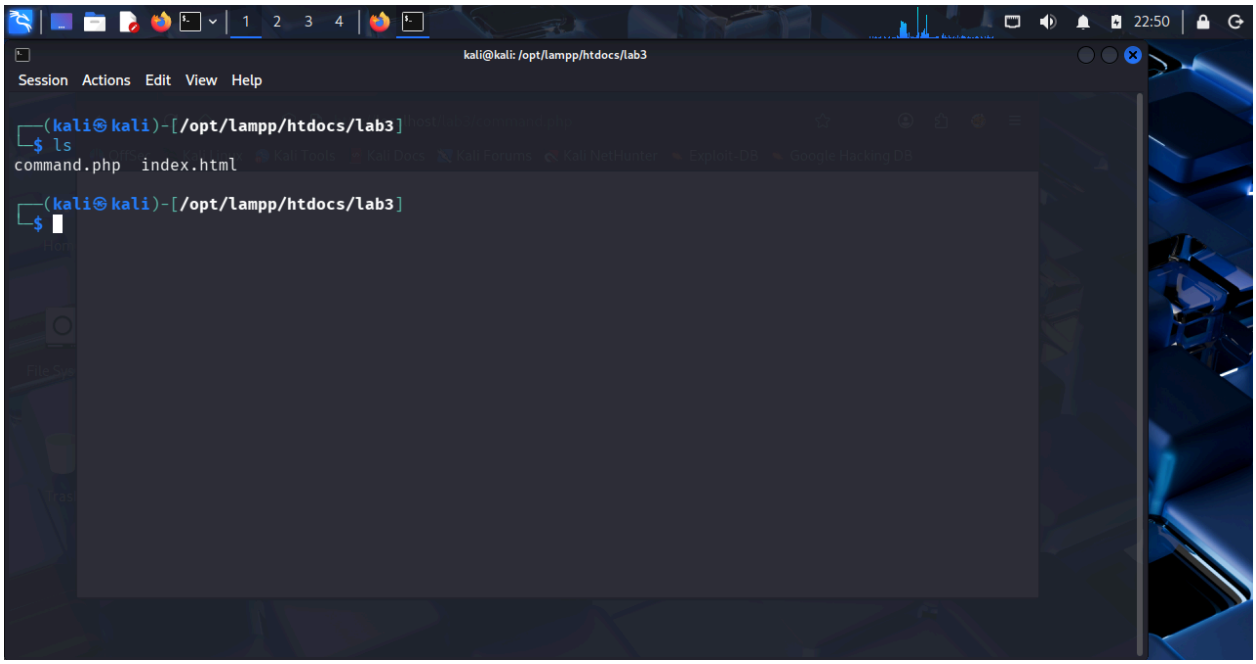
I first created a folder lab3 and created the index.html and command.php in it

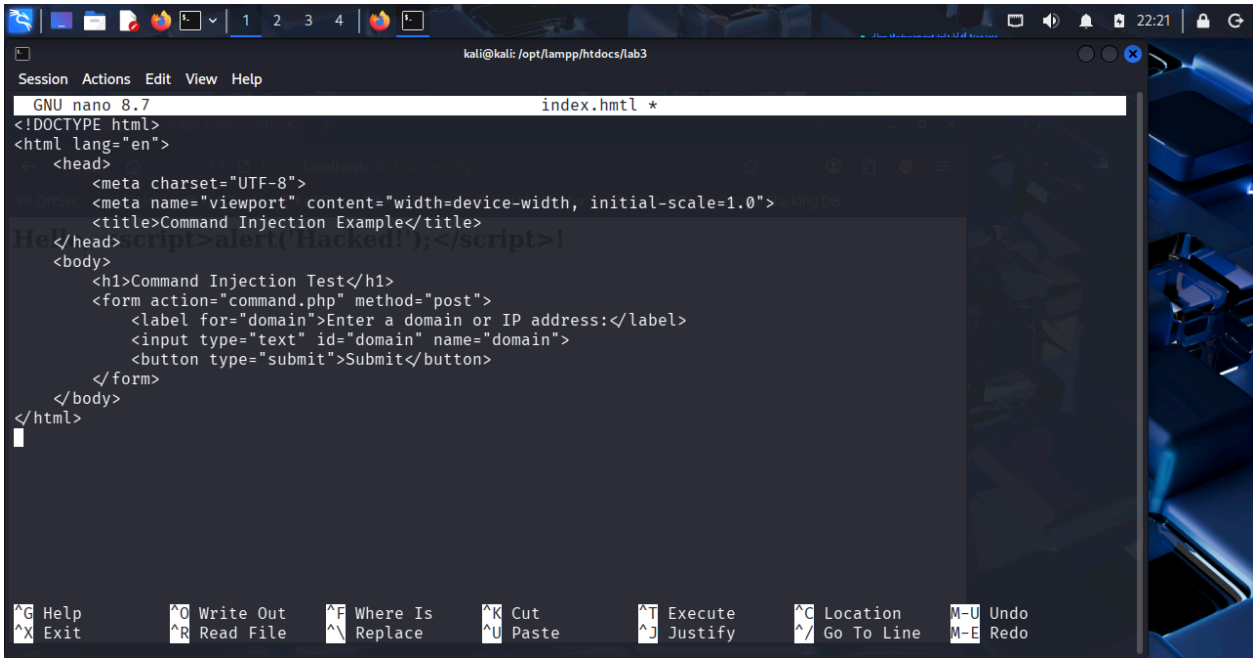
I created a simple PHP application that runs a `ping` command using user input. The input was not validated.

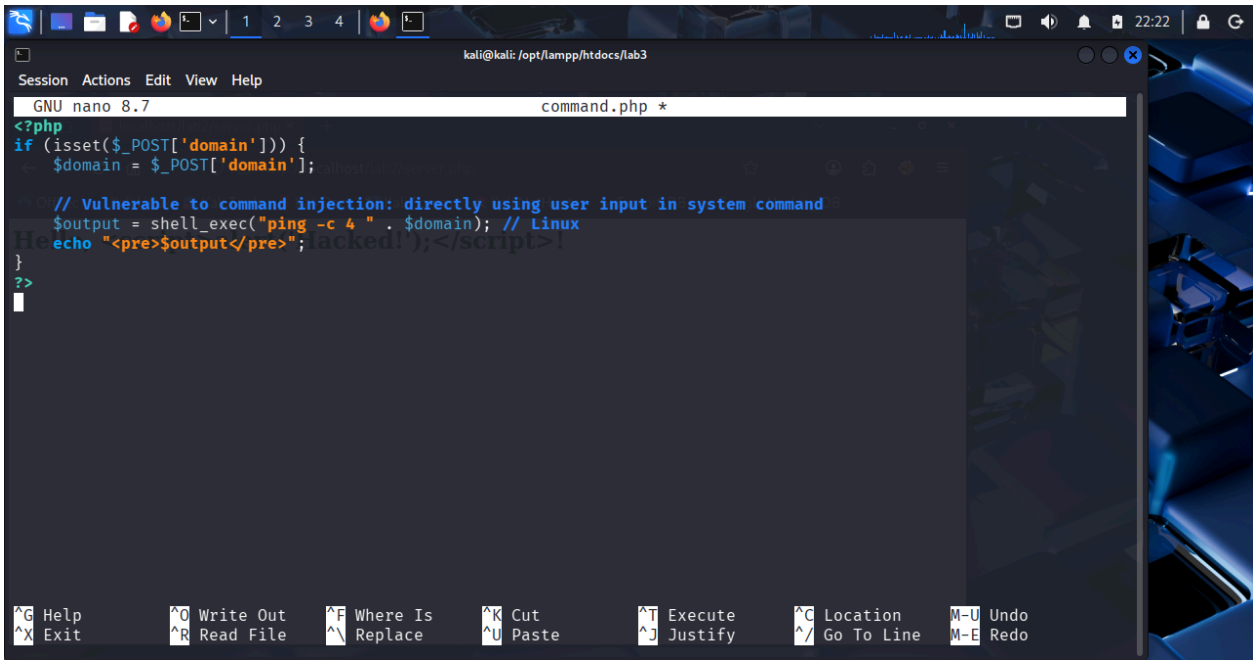
I entered the payload below in the form:

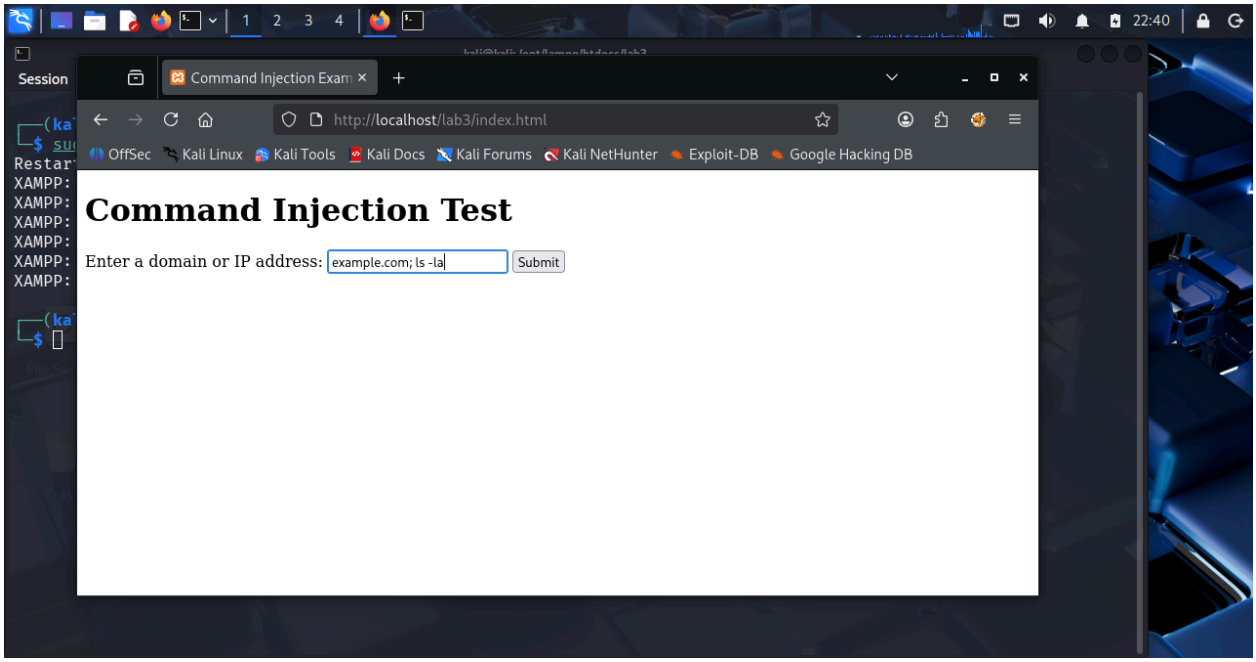
example.com; ls -la

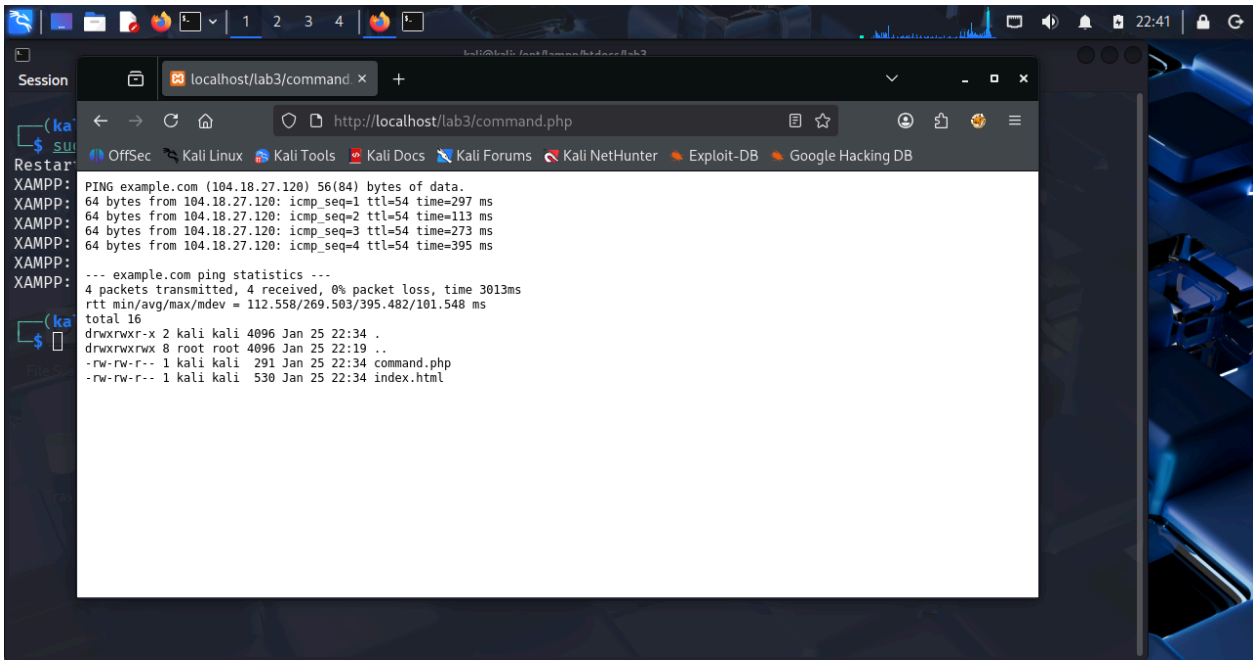
The server executed both the `ping` command and `ls -la`. The directory contents were displayed. This confirmed the application was vulnerable to command injection.




kali@kali: /opt/lampp/htdocs/lab3
Session Actions Edit View Help
GNU nano 8.7 index.hmtl *
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Command Injection Example</title>
</head>
<body>
<h1>Command Injection Test</h1>
<form action="command.php" method="post">
<label for="domain">Enter a domain or IP address:</label>
<input type="text" id="domain" name="domain">
<button type="submit">Submit</button>
</form>
</body>
</html>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo


kali@kali: /opt/lampp/htdocs/lab3
Session Actions Edit View Help
GNU nano 8.7 command.php *
<?php
if (isset($_POST['domain'])) {
$domain = $_POST['domain'];
// Vulnerable to command injection: directly using user input in system command
$output = shell_exec("ping -c 4 " . $domain); // Linux
echo "<pre>$output</pre>";
}
?>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo



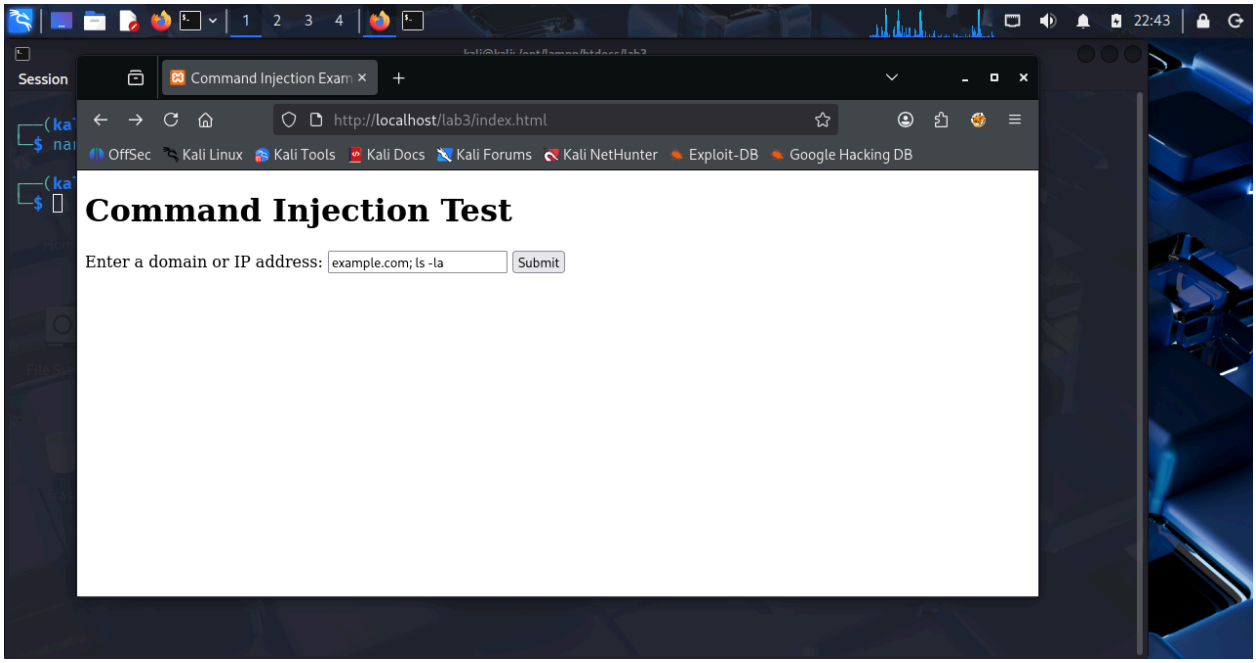


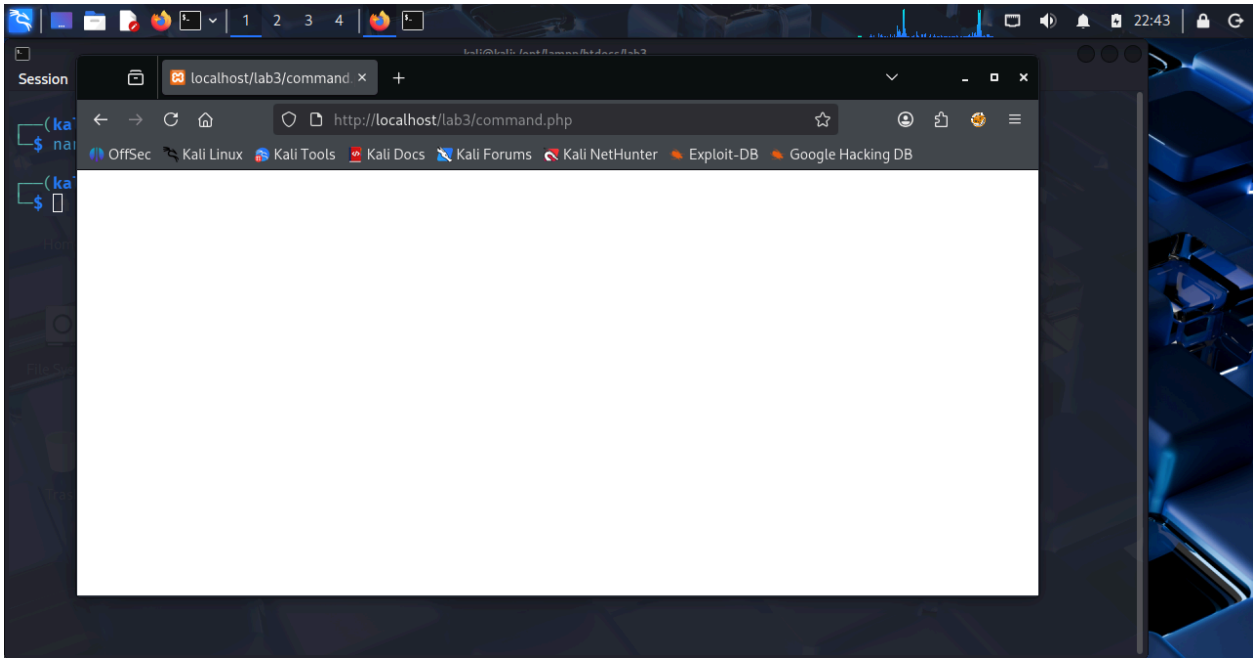
Task 2: Mitigation

I fixed the issue by sanitizing the input using `escapeshellcmd()` before running the command.

I tested the same payload again. This time, only the ping command ran. The extra command was blocked.


kali@kali: /opt/lampp/htdocs/lab3
Session Actions Edit View Help
GNU nano 8.7 command.php *
<?php
if (isset($_POST['domain'])) {
$domain = $_POST['domain'];
// Mitigate the command injection vulnerability
$safe_domain = escapeshellcmd($domain);
// Safely execute the ping command
$output = shell_exec("ping -c 4 " . $safe_domain); // Linux
echo "<pre>$output</pre>";
}
?>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo


Session
Command Injection Exam
http://localhost/lab3/index.html
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Command Injection Test
Enter a domain or IP address: example.com; ls -la Submit



Conclusion

This lab showed how command injection works and how dangerous it can be. It also showed that simple input validation can prevent attackers from running system commands.